ФСП Карьера

Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника Федерации спортивного программирования

Сопроводительная документация к решению
Хакатон «Лидеры цифровой трансформации 2026», задача ФСП

Команда: [название команды]

Содержание

Оглавление обновится при открытии (F9 или «Обновить поле»).

Обзор решения

Платформа подбора ИТ-специалистов с обратной механикой: кандидат проходит адаптивный тест и получает категорию, а работодатель находит категорию и сам приглашает кандидата, сразу указывая вилку зарплаты.

Статус реализации

Подробная трассировка — docs/09-requirements-trace.md.

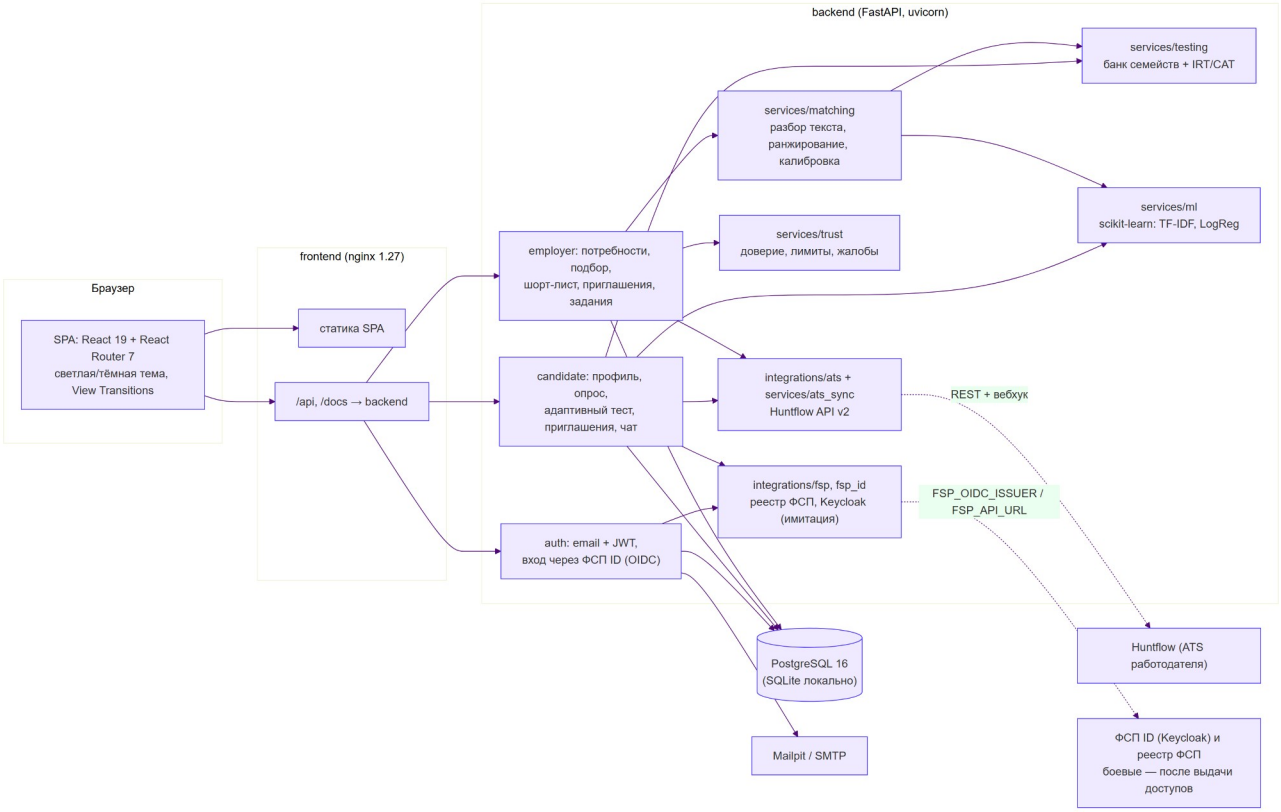
Требование	Статус
Регистрация по email с подтверждением, роли, согласие по 152-ФЗ	готово
Вход и привязка через ФСП ID (OIDC, имитация Keycloak + заготовка подключения к настоящему Keycloak)	готово
8 специализаций, 6 грейдов (Intern … Senior, с Junior+ и Middle+), ИТ-направления	готово
Опрос с самооценкой, самостоятельный выбор грейда (ML-подсказка), адаптивный тест до 20 минут, категория	готово
Неподтверждённый грейд виден со статусом и ниже подтверждённых; кандидат может скрыть профиль до пересдачи	готово
Тест на уровень ниже или выше (уверенным — сразу), кулдаун 90 дней, провал без сжигания попытки	готово
История опроса и теста	готово
Профиль: ручной ввод и разбор PDF	готово (словарь + ML/NLP: специализация, опечатки)
Автогенерация стандартизированного PDF-профиля	готово (обезличенная версия для работодателя)
Приватность, согласие на публикацию, достижения из ФСП ID, «нет истории ФСП»	готово
Все соревнования ФСП равнозначны: число стартов и результат, потолок «явок», свежесть	готово
Потребность свободным текстом, сначала категории, затем объяснимая подборка; новые кандидаты	готово
Требования «обязательно / желательно» с уровнем, настройка модели релевантности	готово
Граф навыков, проверка навыков из резюме коротким тестом	готово
Поиск по банку, шорт-лист, адресное приглашение с вилкой (gross), несколько позиций, оффер и выход	готово
Зарплатные ожидания, фильтр «не ниже моих ожиданий»	готово
Контакты после принятия, отзыв и возврат доступа	готово
Переписка по приглашению	готово
Регулярные задания: эталон проверяет система, открытый ответ — работодатель; адресное «задание для вас»	готово
Сигналы честности теста (уход с вкладки, копирование, быстрые ответы)	готово
Несовершеннолетние 15–17: согласие представителя, отметка «до 18»	готово
Метка доверия компании (ИНН)	готово
Быстрый вход для жюри без регистрации	готово (`DEV_MODE`)
OpenAPI и описание API	готово
Интерфейс: брендбук ФСП, светлая и тёмная темы, анимации и переходы, переключение RU / EN	готово
Импорт вакансий со сторонних площадок	не делаем — по ответу постановщиков не нужен
Защита от фиктивных вакансий: доверие 0–100, лимиты, жалобы с автоприостановкой, вилка-приманка	готово
Интеграция с ATS Huntflow: кандидаты и этапы, вебхук, импорт вакансий	готово (демо-режим без токена)

Структура репозитория

```
fsp-hackathon/
├── README.md                      обзор, запуск, статус по ТЗ
├── docker-compose.yml             Postgres + Mailpit + backend + frontend
├── materials/                     материалы организаторов: ТЗ, ответы на вопросы, шаблон (локально,
в git не входят)
├── docs/
│   ├── README.md                  оглавление документации
│   ├── 01-concept.md              механика подбора, модель релевантности, тестирование, ФСП,
концепции за рамками MVP
│   ├── 02-validation.md           отчёт валидации (генерируется скриптом)
│   ├── 04-deploy.md               развёртывание на сервере
│   ├── 05-architecture.md         архитектура, потоки, нагрузка, безопасность
│   ├── 06-fsp-integration.md      ФСП ID (OIDC/Keycloak) и реестр: схемы, правила учёта достижений
│   ├── 07-api.md, openapi.json    описание API
│   ├── 08-libraries.md            библиотеки и версии
│   ├── 09-requirements-trace.md   соответствие ответам постановщиков
│   ├── ФСП_Карьера_документация.docx  вся документация одним файлом
│   └── ФСП_Карьера_презентация.pptx  презентация для питча
├── backend/                       FastAPI + SQLAlchemy
│   ├── app/
│   │   ├── main.py                точка входа, подключение роутеров
│   │   ├── core/                  конфигурация, БД, JWT (claims в формате Keycloak)
│   │   │   ├── catalog.py         специализации, грейды, отрасли
│   │   │   └── skills.py          граф навыков, синонимы, коэффициенты переноса
│   │   ├── models/entities.py     сущности БД
│   │   ├── api/                   роутеры: auth, public, candidate, employer, integrations (вебхук
ATS); правила видимости в common.py
│   │   ├── services/
│   │   │   ├── testing/           item_bank + item_bank_ext (52 семейства заданий), cat (IRT + CAT),
skill_profile, service
│   │   │   ├── matching/          text_parser (разбор текста), ranking (релевантность),
calibration.json
│   │   │   ├── ml.py              scikit-learn: специализация по тексту, нечёткие навыки, подсказка
грейда
│   │   │   ├── trust.py           доверие к работодателю, лимиты, жалобы
│   │   │   └── ats_sync.py        синхронизация с ATS
│   │   ├── integrations/          fsp.py — реестр ФСП, fsp_id.py — ФСП ID (mock и Keycloak), ats.py
— Huntflow (mock и HTTP)
│   │   ├── services/profile_pdf.py  стандартизированный PDF-профиль
│   │   └── seed/demo.py           демо-данные
│   ├── scripts/validate.py        процедура валидации
│   └── tests/                     сквозные и юнит-тесты
└── frontend/                      React + TypeScript + Vite
    └── src/
        ├── api.ts, ui.tsx, match.tsx   клиент API, дизайн-система (тема, анимации, тосты, чат),
разбивка релевантности
        ├── fspid.tsx              вход через ФСП ID (OIDC)
        ├── i18n.ts                переключение языка RU / EN
        └── pages/
            ├── candidate/         кабинет кандидата (/c)
            └── employer/          кабинет работодателя (/e)
```

1. Функциональная и компонентная архитектура

Компоненты



Слой	Где в коде	Что делает
API	`backend/app/api/`	Роутеры `auth`, `public`, `candidate`, `employer`; правила видимости данных в `common.py`, переписка в `messaging.py`
Тестирование	`backend/app/services/testing/`	Параметрические семейства заданий (`item_bank.py`), 2PL IRT и адаптивный выбор (`cat.py`), профиль навыков, сигналы честности (`service.py`)
Подбор	`backend/app/services/matching/`	Разбор текста вакансии и резюме, модель релевантности из трёх уровней, изотоническая калибровка (`ranking.py`, `calibration.json`)
Интеграции	`backend/app/integrations/`	Реестр ФСП (`fsp.py`: mock и HTTP-клиент), ФСП ID по OpenID Connect (`fsp_id.py`: mock и Keycloak), Huntflow (`ats.py`: mock и HTTP-клиент)
Доверие	`backend/app/services/trust.py`	Оценка доверия к работодателю, суточные лимиты, жалобы и автоприостановка, флаг вилки-приманки
ATS	`backend/app/services/ats_sync.py`, `app/api/integrations.py`	Передача кандидата и этапов в Huntflow, вебхук этапов, импорт вакансий
ML и NLP	`backend/app/services/ml.py`	Классификатор специализации по тексту, нечёткое распознавание навыков, подсказка грейда (scikit-learn)
Данные	`backend/app/models/entities.py`	SQLAlchemy 2. Схема создаётся при старте, недостающие колонки добавляет `ensure_columns()`

Фронтенд	`frontend/src/`	Страницы кандидата и работодателя, `ui.tsx` — дизайн-система (тема, тосты, скелетоны, кольцо релевантности, чат), `fspid.tsx` — вход через ФСП ID, `i18n.ts` — переключение RU / EN

Ключевые потоки

Кандидат. Регистрация по email или вход через ФСП ID → опрос с самооценкой (кандидат сам выбирает одну из 6 ступеней, ML-модель подсказывает) → адаптивный тест (12–22 задания, до 20 минут, варианты генерируются заново, правильный ответ не уходит в браузер) → категория. Уверенный результат — система сразу предлагает тест на ступень выше. Если грейд не подтверждён, кандидат остаётся в выдаче со статусом «не подтверждён» и ниже подтверждённых. Дальше согласие на публикацию → приглашения → переписка → принятие (контакты открываются, доступ можно отозвать) → оффер → выход на работу. Адресные регулярные задания поддерживают актуальность профиля.

Работодатель. Потребность свободным текстом → разбор в специализацию, грейд, стек и вилку (NLP-подсказки по опечаткам) → шаг 1: карточки рекомендованных категорий → шаг 2: объяснимая подборка (6 блоков, покрытие каждого требования, флаги) → шорт-лист → адресное приглашение с вилкой до вычета НДФЛ → переписка → контакты после согласия кандидата. Дополнительно — регулярные задания: с эталоном их проверяет система, открытые оценивает работодатель. Статусы приглашений и кандидаты синхронизируются с Huntflow.

Нагрузка и стенд

- Эндпоинты синхронные и выполняются в пуле потоков uvicorn, БД — PostgreSQL с `pool_pre_ping`.
- Проверка на ноутбуке (SQLite, один процесс uvicorn): 60 запросов к самым тяжёлым эндпоинтам (подборка по потребности, поиск по банку, профиль, приглашения) в 50 параллельных потоков — 0 ошибок, p50 0,7 с, p95 1,4 с.
- GPU и внешние сервисы не нужны: тест, подбор, калибровка и ML-модели считаются на CPU (numpy, scikit-learn; модели обучаются при первом обращении менее чем за 2 секунды). Целевой стенд жюри (32–64 ГБ RAM, ~8 CPU) с запасом покрывает десятки одновременных пользователей.
- Масштабирование: несколько реплик backend за nginx (состояние только в БД), отдельный read-replica для подборки. Одноразовость кода имитированного ФСП ID хранится в памяти процесса. С реальным Keycloak её гарантирует сам Keycloak.

Безопасность и данные

- JWT HS256 с claims в формате Keycloak (`sub`, `email`, `realm_access.roles`). Переход на токены ФСП ID меняет только проверку ключа.
- Пароли — bcrypt. Правильные ответы теста хранятся только на сервере. Ответ после лимита времени не засчитывается.
- Обезличенные карточки: до согласия кандидата видны «Кандидат #N», категория, проверенные навыки и ФСП. Контакты открываются только по принятому приглашению или отклику, и кандидат может отозвать доступ.
- 152-ФЗ: согласие на обработку при регистрации или на экране ФСП ID, отдельное согласие на публикацию, согласие законного представителя для 15–17 лет, до 15 лет публикация запрещена.
- Защита от фиктивных вакансий: оценка доверия, суточные лимиты, жалобы с автоприостановкой (см. 01-concept.md, раздел 5).

- LLM в рабочем контуре не используется (см. 09-requirements-trace.md), поэтому персональные данные не передаются третьим лицам и за рубеж.

2. Механики подбора, категоризации и тестирования

Платформа подбора ИТ-специалистов с **обратной механикой**. Кандидат не рассылает отклики: он один раз проходит обязательный путь (опрос → заявленный грейд → адаптивный тест) и попадает в категорию. Работодатель описывает потребность обычным текстом, видит подходящие категории и конкретных людей с объяснением, почему они в подборке, и сам отправляет приглашение, сразу указывая вилку зарплаты.

Веса критериев оценки определяют приоритеты: подбор 35%, тестирование 25%, остальное 40% в сумме.

1. Механика контакта (критерий 1, 35%)

Элемент	Что делает	Зачем
Потребность свободным текстом	NLP-разбор текста в специализацию, грейд, стек и вилку; работодатель правит результат	Работодатель не обязан знать нашу таксономию
Сначала категории, потом люди	Шаг 1 — карточки рекомендованных категорий (точная, соседний грейд Middle ↔ Middle+, смежные специализации) с числом кандидатов, лучшим процентом, долей ФСП, неподтверждёнными и новыми; шаг 2 — кандидаты выбранной категории	Работодатель сначала выбирает категорию, затем человека, как в модели LHH
Новые кандидаты	Кто появился в категории потребности после последнего просмотра подборки, отмечен «новый»; счётчик в меню	Работодатель возвращается к подборке, не пересматривая её целиком
Ранжирование с объяснениями	Общий процент, откалиброванная вероятность, 6 блоков, покрытие каждого требования и флаги	Требование ТЗ к объяснимости выдачи; один навык не закрывает другой
Настраиваемый подбор	Веса блоков, строгость, пороги, жёсткие фильтры с подсчётом отсеянных	Работодатель сам решает, что для него критично
Обезличенная карточка	До согласия кандидата видны «Кандидат #N», категория, проверенные навыки, ФСП; ФИО и контакты скрыты	Правило 2.2.2 ТЗ; снижает дискриминацию
Зарплатный сигнал без раскрытия	Работодатель видит «ожидания в вилке / выше примерно на X%», но не саму сумму	Кандидат не теряет позицию на переговорах, работодатель не тратит приглашение впустую
Фильтр «не ниже моих ожиданий»	Кандидат может запретить приглашения с вилкой ниже своих ожиданий; работодатель получает понятный отказ при отправке	Кандидат не тратит время на заведомо неподходящие предложения
Приглашение с обязательной вилкой	Позиция, вилка от и до в рублях, описание, способ связи; привязка к вакансии необязательна	Правило 2.2.1 ТЗ
«Почему вас выбрали»	Кандидат видит снимок соответствия: процент, блоки, статус каждого навыка	Прозрачность для обеих сторон; понятно, какой навык подтвердить
Лимит приглашений	В сутки: новая компания — 15, средний уровень доверия — 40, высокий — 60; компания на проверке — 0	Приглашение остаётся ценным, но лимит не мешает нормальному найму
Несколько позиций одному кандидату	Можно предложить разные позиции; дубль активного приглашения на ту же позицию запрещён	Работодатель не теряет кандидата, если у него две подходящие роли
Срок жизни и статусы	sent → viewed → accepted → **offer** → **hired**; declined (с причиной) / expired (14 дней) / withdrawn	Правило 2.2.3 ТЗ; оффер и выход видны кандидату и синхронизируются с ATS
Доверие к работодателю	Кандидат видит оценку 0–100 и сигналы: ИНН, домен почты, доля	Защита от недобросовестных работодателей (раздел 5)

	принятых, ответы после принятия, офферы, жалобы	
Жалобы	Кандидат может пожаловаться на приглашение; 3 жалобы разных кандидатов за 30 дней приостанавливают приглашения и вакансии компании	Фиктивные вакансии и «сборщики резюме» быстро теряют доступ
Шорт-лист	Отобранные кандидаты сохраняются при смене фильтров	«Уточнение без потери подборки» из ТЗ
Отклик (вспомогательный путь)	Кандидат с категорией может откликнуться на опубликованную вакансию, контакты при этом раскрываются	Дополнительный функционал, 7%
Переписка по приглашению	Обе стороны пишут друг другу до и после принятия, контакты при этом не раскрываются	Удобнее, чем просто обменяться контактами
Отзыв доступа к контактам	После принятия кандидат может отозвать и вернуть доступ; работодатель снова видит обезличенную карточку	Контроль над ПДн у кандидата
Неподтверждённый грейд не прячется	Кандидат виден с заявленным грейдом и пометкой «не подтверждён», ниже подтверждённых (штраф ×0.6). Кандидат может сам скрыть профиль до пересдачи	Не выпадает из оборота (ответ постановщиков), выбор остаётся за кандидатом
Метка доверия компании	ИНН с корректной контрольной суммой → «ИНН проверен» в приглашении	Добровольная, основной сценарий не блокирует
Несовершеннолетние 15–17 лет	Публикация только с согласием законного представителя, работодатель видит «до 18, только лёгкий труд»	152-ФЗ, ст. 63 ТК РФ

Модель релевантности

Релевантность считается в три уровня, и каждый уровень показывается работодателю отдельно.

Уровень 1. Требования. Потребность — это список навыков, у каждого есть важность (обязательно или желательно) и уровень (базовый, уверенный, экспертный). Требования предзаполняются из текста вакансии: «будет плюсом Docker» даёт «желательно», «глубокое знание SQL» даёт «эксперт». Работодатель их правит. По каждому требованию считается **покрытие** — вероятность, что кандидат владеет навыком на нужном уровне — и **уверенность** в этой оценке.

Доказательства берутся из нескольких источников с разным доверием:

Источник	**Как учитывается**
Тест	P(θ_навык ≥ порог уровня) по апостериорной оценке навыка; опорный источник, его вес равен его уверенности
Регулярные задания	Оценка работодателя от 1 до 5 по навыкам задания, доверие 0.8
ФСП	Место в соревновании (уровень мероприятия не взвешивается — все соревнования ФСП равнозначны), доверие 0.7
Резюме	Не больше 0.3 и только если нет прямого объективного подтверждения

- **Граф навыков.** Подтверждение переносится между навыками: FastAPI → Python ×0.8, Python → FastAPI ×0.5, Django ↔ FastAPI ×0.35, плюс явные связи (TypeScript ↔ JavaScript, Kotlin ↔ Java).
- **Подкреплённое заявление.** Кандидат указал в резюме «PostgreSQL» и прошёл тест по SQL: заявление подкреплено проверенным навыком-родителем и засчитывается ×0.9.

- **Тест главнее остальных источников.** `покрытие = уверенность_теста × тест + (1 − уверенность_теста) × max(тест, другие)`. Уверенный провал теста не перекрывается ни ФСП, ни резюме, а при малом числе заданий другие источники честно добавляют информацию.
- **Статус требования.** «Подтверждён»; «не закрыт» (прямая проверка показала, что уровня нет, или данных нет совсем); «не подтверждён напрямую» (только резюме или косвенный тест). Для последнего кандидату предлагается проверка навыка.

Уровень 2. Блоки. Роль (категория из теста: своя — 1.0, соседняя ступень — 0.6, через ступень — 0.25), Стек (покрытие требований), Уровень (запас θ над порогом грейда), Подтверждённость (доля требований, закрытых объективными источниками), Условия (зарплата и формат), Актуальность. Каждый блок — число от 0 до 1 с текстовым объяснением. **ФСП — отдельный блок-бонус.**

Уровень 3. Итоговый процент.

```
база   = степенное_среднее(блоки, веса, p)        p = 1 — сумма, 0 — геометрическое, < 0 — к минимуму
итог   = база + бонус_ФСП · сила_ФСП · (1 − база)  бонус никогда не уменьшает балл
итог  *= штраф_must ^ (число проваленных обязательных навыков)
итог  *= штраф_непроверенного ^ (число обязательных навыков без прямого подтверждения)
```

Рядом с процентом показывается **откалиброванная вероятность** («≈ 63% таких кандидатов подходят»). Её даёт изотоническая регрессия, обученная на валидационных данных. На реальных данных она доучивается по действиям работодателей.

Всё настраивается в потребности: веса блоков, строгость p, бонус ФСП, вес желательных навыков, порог и штраф must-have, штраф за неподтверждённый навык, учёт уверенности в сортировке. Условия по умолчанию мягкие: кандидат остаётся в выдаче с флагом. Любое из них можно сделать жёстким фильтром (обязательные навыки, вилка, формат, точный грейд, только ФСП), и выдача покажет, сколько кандидатов скрыл каждый фильтр.

Значения по умолчанию выбраны по валидации (раздел 6 в 02-validation.md): геометрическое среднее, порог must-have 0.5 (покрытие — это вероятность, ниже 0.5 «скорее нет»), штраф 0.6 за проваленный обязательный навык, без учёта уверенности в сортировке: на синтетике он снижал nDCG.

Сортировка: сначала точность категории, внутри неё итоговый балл. Так выполняется требование «внутри категории выше те, у кого сильнее подтверждённый профиль».

Проверка навыков из резюме

Навык, указанный в резюме, но не проверенный тестом, весит мало. Кандидат видит список своих навыков со статусами и количеством вакансий, которым они нужны, и может пройти **короткую проверку навыка** (4–6 заданий). Проверка адаптивна именно по этому навыку и не меняет грейд. Повтор возможен через 14 дней. Валидация показывает, зачем это нужно: среди «блестящих» кандидатов без одного обязательного навыка в топ-10 попадают 5–7% тех, кто проверял навык, и 22–28% тех, кто не проверял.

2. Механика тестирования (критерий 2, 25%)

Справочник. 8 специализаций: Backend, Frontend, Mobile, Data Science / ML, Аналитик (системный и продуктовый), QA, DevOps, ИБ. 6 ступеней грейда: Intern, Junior, Junior+, Middle, Middle+, Senior. Грейд — уровень навыков, подтверждённый тестом, а не стаж. «Отрасль» в опросе — ИТ-направление (разработка, данные, аналитика, качество, инфраструктура, ИБ).

Опрос. Направление и специализация, частые задачи и опросник самооценки из шести утверждений по специализации (шкала: не делал / с помощью / самостоятельно / учу других). Опросник не присваивает грейд: он питает ML-подсказку грейда, которую кандидат видит при выборе ступени.

Проблема из ТЗ: единый набор заданий утекает, а генерация уникальных заданий LLM не гарантирует сопоставимую сложность.

Наш подход: параметрические семейства заданий, откалиброванные по IRT, плюс адаптивное тестирование (CAT).

1. **Семейство** — это шаблон задачи с генератором. Каждый вариант заново генерирует числа, имена, код и порядок ответов, правильный ответ вычисляется программно. Примеры: вывод кода со срезами, LEFT JOIN по заданным данным, число попаданий в LRU-кэш, порядок event loop, асимптотика случайной вложенности циклов, подсети, cron.
2. **Калибровка**: у семейства есть параметры 2PL — трудность *b* и дискриминативность *a*. Все варианты семейства изоморфны, поэтому сложность сопоставима между кандидатами. Именно это LLM-генерация «на лету» не обеспечивает.
3. **Адаптивность**: старт с априорной оценкой по заявленному грейду, затем задание с максимумом информации Фишера в текущей оценке θ; выбор случайный из топ-4 для контроля экспозиции; не более двух заданий подряд на один навык. Остановка при SE ≤ 0.30 либо на 22 заданиях (минимум 12); общий лимит теста — 20 минут.
4. **Решение о грейде** принимается по апостериорной вероятности P(θ ≥ порог), а не по проценту верных:
   - P ≥ 0.5 (медиана апостериорного распределения не ниже порога) — грейд подтверждён. Для полос шириной 0.5 это несмещённое решение; порог 0.6 давал систематическое занижение (подобрано по валидации);
   - дополнительно P(θ ≥ следующий порог) ≥ 0.8 — «подтверждён уверенно»: система сама предлагает отдельный тест на ступень выше, и он доступен сразу, без ожидания 90 дней. Автоматического повышения нет;
   - иначе «не подтверждён». Грейд **не понижается принудительно**, предлагается тест на уровень ниже (доступен сразу). Если подтверждённого грейда ещё нет, кандидат **не скрывается**: в выдаче он виден с заявленным грейдом и статусом «не подтверждён», ниже подтверждённых кандидатов своей категории.
5. **Правила**: грейд кандидат выбирает сам (ML-модель только подсказывает по стажу, стеку, ФСП и опроснику); смена подтверждённого грейда не чаще раза в 90 дней; повтор того же уровня после неудачи через 30 дней, тест на уровень ниже — сразу, попытка не сгорает; ответ после истечения лимита времени не засчитывается; правильный ответ никогда не уходит на клиент.
6. **Вес «знаниевых» заданий.** У семейств с малым числом уникальных формулировок (меньше 250 из 300 вариантов) ответ легко слить, поэтому в оценке θ их дискриминативность умножается на 0.5. На атаке «слитый ключ» это снизило долю слабых кандидатов, прошедших на Middle, с 37% до 16% при почти неизменной точности грейда.
7. **Честность прохождения**: копирование условия заблокировано; уходы с вкладки, попытки копирования и подозрительно быстрые ответы фиксируются и показываются работодателю сводкой («тест пройден чисто / есть сигналы»). На оценку θ они не влияют, чтобы случайный уход с вкладки не стоил грейда.
8. **Роль LLM** — только офлайн: LLM предлагает новые *семейства* (шаблон и генератор). Семейство попадает в банк как pretest-задание без влияния на грейд, калибруется на ответах и только потом становится рабочим.
9. **Мониторинг утечки** (следующий этап): если доля верных ответов по семейству растёт быстрее, чем предсказывает модель при текущих θ отвечающих, это сигнал на ротацию семейства (дрейф параметров).
10. **Профиль навыков**: по каждому навыку считается своя оценка θ_навык с неопределённостью, стянутая к общему уровню кандидата (partial pooling): мало заданий — оценка близка к общему уровню и уверенность низкая. Адаптивный выбор заданий учитывает покрытие навыков: пока по навыку меньше двух заданий, его семейства получают приоритет. У каждого ключевого навыка в банке есть задания нескольких уровней сложности, иначе провал средних заданий не отличает «не знает основ» от «не дотягивает до middle».

11. **Тест под вакансию**: категорийный тест одинаков по структуре, вакансия только смещает приоритет семейств на свой стек (вкладка «Как тестируем кандидатов» показывает состав и пример варианта).

Регулярные короткие задания от работодателей обновляют актуальность профиля и дают работодателю свежий сигнал. Задания с эталоном проверяет система, открытые («предложите подход») оценивает работодатель по шкале от 1 до 5. Задания адресные: система сама выбирает для кандидата одно «задание для вас» — по навыку, где в его профиле меньше всего данных, — и показывает счётчик новых заданий в меню.

3. Валидация (часть критерия 2)

Эталонной разметки нет, поэтому мы генерируем популяцию с **известной истиной** (способность θ, специализация, навыки), 30% кандидатов завышают грейд, 30% резюме приукрашены. Эта популяция проходит через **реальный код** тестирования и подбора. Отчёт: 02-validation.md, команда `python -m scripts.validate`.

У каждого кандидата своя способность по каждому навыку (общий θ плюс сдвиг навыка), поэтому эталон «владеет навыком на требуемом уровне» согласован с тем, как кандидат отвечает на задания.

Измеряем:

- по тесту: точность грейда и матрицу ошибок, повторное прохождение (каппа Коэна), точечно-бисериальную корреляцию по семействам, атаку «слитый ключ» в сравнении с фиксированным тестом, устойчивость к ошибке калибровки;
- по подбору: precision@10, strict precision@10 и nDCG@10 в сравнении с keyword-бейзлайном и «оракулом»; долю кандидатов в топ-10 без обязательного навыка; **стресс-тест компенсации** («блестящие» кандидаты без одного обязательного навыка) отдельно для проверивших и не проверивших навык; **калибровку процента** (ECE до и после изотонической регрессии, обучение и проверка на разных половинах кандидатов);
- **абляцию**: одни и те же данные прогоняются через разные способы агрегации (сумма, геометрическое среднее, штрафы, учёт уверенности, строгий режим), чтобы показать вклад каждого механизма и обосновать значения по умолчанию.

Эталонная разметка — истина по построению: у каждого синтетического кандидата известны уровень, специализация и навыки, а релевантность пары «потребность — кандидат» задаётся правилами эксперта, независимыми от ранжирования. Это и есть «эталонная разметка команды», которую допускают постановщики.

Следующий шаг: 10–20 живых респондентов с известным уровнем (коллеги и участники ФСП) проходят тест дважды. Это даст реальную оценку повторного прохождения и сравнение с экспертной оценкой грейда.

4. ФСП

Профиль связывается с ФСП ID через вход по OIDC (Authorization Code). В демо работает имитация Keycloak, в продакшене — Keycloak ФСП (переключается переменными окружения). `sub` токена и есть ФСП ID, поэтому привязать чужой ID нельзя. Схемы потоков и правила учёта достижений — 06-fsp-integration.md.

Все соревнования ФСП учитываются одинаково по престижу, но учитывается профильность дисциплины для специализации (идея постановщиков: ИБ-специалисту CTF релевантнее робототехники; коэффициент от 0.6 до 1.0). Уровень мероприятия не взвешивается, считаются число стартов и результативность (победа > призовое место > топ-10 > участие), вклад одних участий ограничен, свежие результаты весят больше. Грейд достижения не меняют. Предлагаемый контракт реестра и данные, которые полезно показывать работодателю: мероприятие, дисциплина, уровень (региональный, всероссийский, международный), дата, место

и число участников, роль в команде, навыки, ссылка на протокол. Случай «истории нет» обрабатывается явно.

5. Защита от фиктивных вакансий и интеграция с ATS

Верификация компании на этапе MVP не обязательна (п. 2.2.4 ТЗ), поэтому защита строится на поведении: каждое действие работодателя оставляет след, который видит кандидат. Реализовано (`app/services/trust.py`):

- **Метки доверия:** ИНН с корректной контрольной суммой; почта компании на домене её сайта.
- **Оценка доверия 0–100** из сигналов: возраст аккаунта, доля принятых приглашений за 90 дней, доля ответов в переписке после принятия (защита от «гостинга»), офферы и выходы на работу, жалобы.
- **Лимит приглашений в сутки** зависит от уровня: новая компания 15, средний уровень 40, высокий 60.
- **Жалобы кандидатов** (сбор резюме, просьба об оплате, несовпадение условий, спам, некорректное общение): 3 жалобы разных кандидатов за 30 дней переводят компанию в статус «на проверке» — приглашения и вакансии приостанавливаются. В демо так выглядит компания «Карьерный Старт Плюс».
- **Вилка-приманка:** если верх вилки в 1.8 раза выше медианы ожиданий кандидатов категории, кандидат видит предупреждение.

Следующий этап: сверка ИНН с ЕГРЮЛ/ФНС, ручная модерация жалоб, эскроу-вилка (вилку из принятого приглашения нельзя снизить на первом интервью без объяснения).

Интеграция с ATS — Huntflow API v2 (`app/integrations/ats.py`, `app/services/ats_sync.py`). Работодатель подключает аккаунт в профиле компании (без токена — демо-режим без сети):

- платформа → ATS: когда контакты раскрыты (принято приглашение или отклик), кандидат создаётся в Huntflow и добавляется на вакансию; оффер, выход и отказ передаются этапом;
- ATS → платформа: вебхук `/api/integrations/huntflow/webhook/{secret}` о смене этапа («Оффер», «Вышел на работу», «Отказ») обновляет статус приглашения, кандидат видит его у себя; переходы только вперёд по воронке;
- вакансии из ATS импортируются черновиками потребностей. Чужие площадки не парсятся — это вакансии самого работодателя;
- журнал синхронизации виден работодателю; персональные данные уходят в ATS только после раскрытия контактов.

6. ML и NLP

Без GPU и внешних API, на scikit-learn (`app/services/ml.py`):

- **Специализация по тексту резюме** — TF-IDF символьных n-грамм и логистическая регрессия; подсказка в опросе и при разборе PDF и описания вакансии. Категорию модель не присваивает.
- **Нечёткое распознавание навыков** — опечатки и транслит («Постгрес», «Кубернетес», «Typscript»); найденное помечается «распознано по сходству», работодатель добавляет навык одним нажатием.
- **Подсказка грейда** — мультиклассовая логистическая регрессия по стажу, ширине стека, призовым местам ФСП и опроснику самооценки; кандидат видит распределение вероятностей по ступеням.
- **Калибровка процента релевантности** — изотоническая регрессия (раздел 1).

Качество на отложенной синтетике — раздел 7 в 02-validation.md. Облачные LLM допустимы, но в обязательный путь не входят: персональные данные никуда не передаются, стенд проверки работает без GPU.

3. Процедура валидации и её результаты

Сгенерирован `python -m scripts.validate`. Популяция с известной истинной способностью θ, специализацией и навыками; 30% кандидатов завышают грейд, 30% резюме «приукрашены».

1. Точность присвоения грейда

- Кандидатов: **1200**, в среднем заданий на кандидата (с учётом пересдач по правилам): **30.0**
- Шкала из 6 грейдов (Intern, Junior, Junior+, Middle, Middle+, Senior) с полосами шириной 0.5 по θ
- Итоговый грейд совпал с истинным: **66.5%**, в пределах ±1 ступени: **97.5%**
- Совпала базовая ступень (Junior/Junior+ → Junior, Middle/Middle+ → Middle): **79.3%**
- Без «пограничных» кандидатов (|θ − порог| > 0.3, n = 257): точность **92.2%**
- Завышение (грейд выше истинного): **11.1%**
- Точность оценки θ: RMSE = **0.49**, корреляция с истиной r = **0.91**

Матрица ошибок (строки — истинный грейд, столбцы — присвоенный):

истина \ итог	intern	junior	junior_plus	middle	middle_plus	senior	—
intern	178	22	0	0	0	0	0
junior	56	91	22	0	0	0	0
junior_plus	10	56	113	20	0	0	0
middle	0	6	53	137	35	2	0
middle_plus	0	0	7	41	93	32	0
senior	0	0	0	5	35	186	0

2. Согласованность при повторном прохождении (test-retest)

- Тот же итоговый грейд при независимом повторе с другими заданиями: **61.2%**
- Без пограничных кандидатов: **90.3%**
- В пределах ±1 ступени: **96.4%**; та же базовая ступень: **75.0%**
- Каппа Коэна: **0.53**

3. Дискриминативность заданий

Точечно-бисериальная корреляция ответа с истинной способностью (> 0.3 — хорошо):

семейство	ответов	доля верных	r_pb
an_bpmn_paths	205	0.39	0.57
ml_prec_recall	345	0.46	0.55
an_retention	224	0.59	0.52
an_link_table	314	0.52	0.52
algo_fib_calls	761	0.21	0.51
sec_vuln_class	348	0.63	0.50
an_ab_uplift	438	0.35	0.50
js_array	382	0.56	0.50
an_funnel	290	0.54	0.49
sql_where_count	1187	0.70	0.48
an_pp_vs_pct	183	0.47	0.48
sql_having	1195	0.37	0.48
net_subnet	734	0.46	0.48
ci_cron	386	0.53	0.48
sd_cache_latency	556	0.36	0.48
kt_collections	420	0.54	0.47
qa_combinations	422	0.58	0.47
android_lifecycle	210	0.30	0.47
sql_left_join	1268	0.45	0.47
py_async_order	286	0.36	0.47
an_arpu	284	0.50	0.46
an_requirement_type	176	0.70	0.45
algo_complexity	1755	0.59	0.45

sql_order_limit	1251	0.59	0.45
algo_lru	1002	0.23	0.45
qa_boundary	700	0.62	0.45
js_event_loop	382	0.33	0.45
prob_at_least_one	791	0.28	0.45
kt_coroutines	420	0.29	0.45
css_specificity	382	0.52	0.44
linux_chmod	1156	0.56	0.44
py_mutable_default	598	0.46	0.44
algo_binsearch	1344	0.46	0.44
css_box	382	0.58	0.44
py_dict_comp	1013	0.59	0.43
mob_pagination	683	0.62	0.42
mob_frame_budget	420	0.42	0.42
react_batching	382	0.26	0.40
http_url	2160	0.70	0.40
sec_bruteforce	348	0.33	0.40
kt_null_safety	420	0.63	0.40
py_slice	1262	0.79	0.39
sec_crypto_kind	174	0.65	0.39
sd_capacity	597	0.17	0.39
mob_dp_px	420	0.72	0.38
sec_xor	348	0.28	0.38
swift_optional	420	0.60	0.38
http_status	1145	0.66	0.38
sec_auth_log	348	0.45	0.37
stat_median	444	0.69	0.34
js_coercion	191	0.61	0.32
sec_port_service	174	0.68	0.27

4. Устойчивость к распространению заданий

- Атака «слитый ключ»: собраны ответы из 200 прохождений. Доля заданий атакующего, совпавших со слитыми: **15.38%**
- Слабый кандидат (θ = −0.8) с этим ключом подтвердил Middle: **15.7%**; без ключа — **4.3%**; при фиксированном тесте с утёкшим ключом — **100%**
- Максимальная экспозиция семейства в пуле Backend (16 семейств): **100%** прохождений
- Уникальных вариантов на семейство (из 2000 сидов):

py_slice: 2000, py_dict_comp: 1653, py_mutable_default: 1990, py_async_order: 1752, algo_complexity: 1544, algo_binsearch: 2000, algo_lru: 2000, algo_fib_calls: 371, sql_left_join: 1926, sql_having: 2000, http_status: 8, sd_capacity: 1986, js_event_loop: 1994, js_coercion: 10, css_specificity: 1996, react_batching: 1827, stat_median: 2000, ml_prec_recall: 1974, prob_at_least_one: 120, qa_boundary: 1070, qa_combinations: 1093, linux_chmod: 503, net_subnet: 1174, ci_cron: 1009, sql_where_count: 2000, sql_order_limit: 2000, http_url: 2000, js_array: 2000, css_box: 1923, sd_cache_latency: 144, mob_dp_px: 40, kt_null_safety: 260, swift_optional: 1068, kt_collections: 2000, mob_frame_budget: 2000, android_lifecycle: 5, kt_coroutines: 998, mob_pagination: 852, an_requirement_type: 9, an_retention: 1755, an_funnel: 2000, an_arpu: 1858, an_pp_vs_pct: 48, an_ab_uplift: 216, an_link_table: 1501, an_bpmn_paths: 24, sec_port_service: 8, sec_crypto_kind: 9, sec_vuln_class: 20, sec_auth_log: 2000, sec_bruteforce: 60, sec_xor: 1962

Семейства с < 50 уникальными формулировками (знаниевые, низкая защита от утечки): http_status, js_coercion, mob_dp_px, android_lifecycle, an_requirement_type, an_pp_vs_pct, an_bpmn_paths, sec_port_service, sec_crypto_kind, sec_vuln_class

5. Робастность к ошибке калибровки

- Параметры a и b заданий случайно смещены на ±0.3: точность грейда **63.8%** (без смещения 66.5%)

6. Качество подбора

12 потребностей (обязательные и желательные навыки) × 900 кандидатов. Эталон считается по истинным свойствам: релевантен, если совпадает специализация, грейд ±1 и обязательные навыки истинно освоены на требуемом уровне (уровень по навыку = общий θ + сдвиг навыка).

Сравнение способов агрегации

вариант	p@10	strict p@10	nDCG@10	провален must-have в топ-10	стресс-тест: доля в топ-10	из проверивших навык попали в топ	из не проверивших попали в топ	завысили грейд в топ-10
Взвешенная сумма (прошлая версия)	0.82	0.38	0.56	24%	22%	8%	5%	3.5
Геометрическое среднее	0.83	0.39	0.58	25%	20%	8%	5%	3.8
+ штраф за must-have, порог 0.4	0.83	0.41	0.58	25%	15%	6%	5%	3.8
+ порог must-have 0.5 (по умолчанию)	0.82	0.41	0.58	24%	13%	5%	5%	3.8
+ сортировка с учётом уверенности (α = 0.5)	0.81	0.41	0.59	24%	16%	6%	7%	3.9
Строгий режим (p = −1)	0.83	0.42	0.59	25%	18%	7%	5%	3.8
Строгий + обязательные только подтверждённые	0.83	0.42	0.59	25%	18%	7%	5%	3.8
Keyword-бейзлайн (резюме + заявленный грейд)	0.46	0.20	0.26	60%	0%	0%	0%	6.2
Оракул (знает истину — потолок)	1.00	0.88	1.00	1%	—	—	—	4.8

«Провален must-have» — доля кандидатов в топ-10, которые на самом деле не владеют хотя бы одним обязательным навыком на требуемом уровне.

«Стресс-тест компенсации» — в базу для каждой потребности добавлены 30 «блестящих» кандидатов: сильный тест, 1 место ФСП (по другим навыкам), свежий профиль, ожидания в вилке и навык в резюме, но один обязательный навык они на самом деле не знают. Показана доля таких кандидатов в топ-10: это прямая проверка, компенсируют ли сильные стороны отсутствие обязательного навыка (идеал — 0%).

Калибровка процента релевантности

Пары «кандидат — потребность»: 3454. Изотоническая регрессия обучена на половине кандидатов, проверена на другой половине. ECE — средняя разница между обещанной и фактической долей релевантных.

- Сырой балл как вероятность: ECE = **0.098**
- После калибровки: ECE = **0.010**

корзина	пар	обещано	фактически
0.0–0.1	1339	2%	2%
0.2–0.3	248	25%	23%
0.4–0.5	21	43%	43%
0.5–0.6	45	60%	62%
0.6–0.7	26	60%	65%
0.8–0.9	31	85%	77%
0.9–1.0	30	93%	90%

Таблица калибровки сохраняется в `app/services/matching/calibration.json` и используется в выдаче (поле `probability`). На реальных данных она доучивается по действиям работодателей.

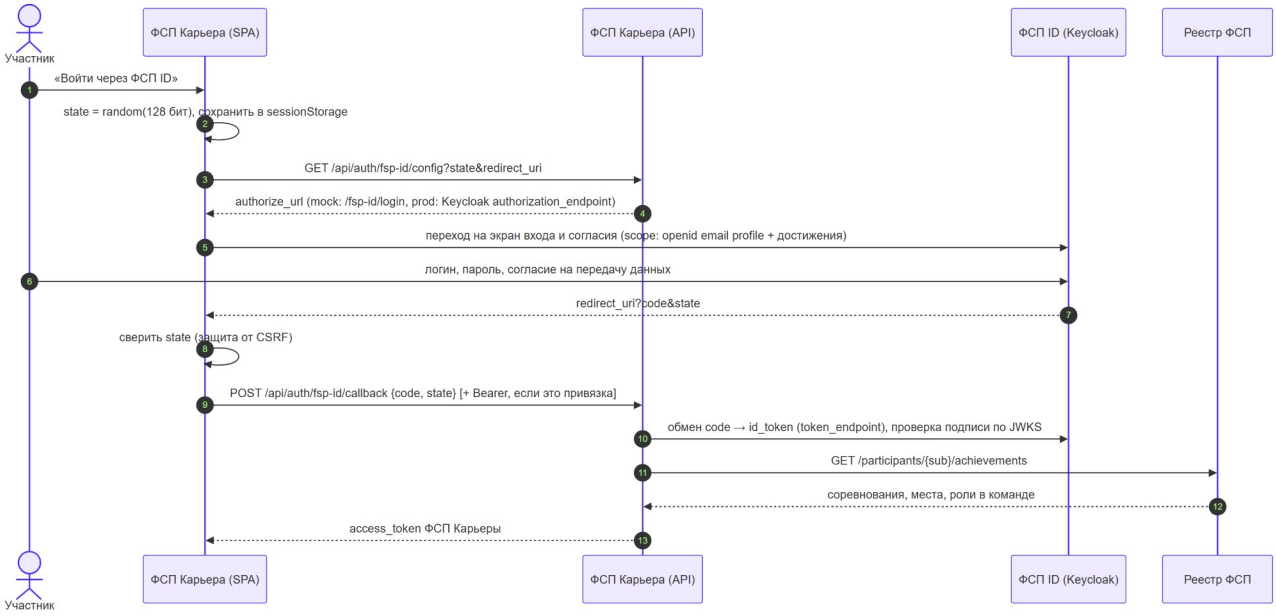
7. ML и NLP-подсказки (отложенная синтетика)

- Классификатор специализации по тексту резюме (TF-IDF + логистическая регрессия), 480 текстов: точность **99.8%**, верная в топ-2 **100.0%**
- Подсказка грейда по профилю и опроснику, 2000 профилей: точно **55.6%**, ±1 ступень **97.5%**. Это подсказка до теста; грейд присваивает только тест (раздел 1)
- Нечёткое распознавание навыков (опечатки, транслит): найдено **10 из 10**

4. Интеграция с реестром ФСП и ФСП ID

Доступа к настоящему ФСП ID (Keycloak) и к API реестра на хакатоне нет — ФСП его не выдаёт. По ответам постановщиков достаточно имитировать подключение и заложить скелет будущей интеграции на своих данных. Ниже — что сделано и как это переключится на настоящий ФСП ID, когда ФСП откроет доступ.

1. Вход и привязка через ФСП ID (OpenID Connect, Authorization Code)



- **Ключ привязки — `sub` из id_token.** Кандидат не вводит ID вручную, поэтому привязать чужой профиль нельзя. Один ФСП ID — один профиль (иначе 409).
- **Три сценария в одном callback:** вход по уже привязанному ФСП ID; вход по email из ФСП ID (профиль связывается); создание нового кандидата (email подтверждён на стороне ФСП ID, согласие на обработку ПДн дано на экране согласия). Если пользователь уже вошёл, тот же поток привязывает ФСП ID к текущему профилю.
- **Имитация (`MockFspIdProvider`).** Экран входа `/fsp-id/login` рисует фронтенд и явно подписан как имитация. Код — короткоживущий (120 с) одноразовый JWT, внутри него `state`. Неверный пароль, чужой `state` и повторное использование кода отклоняются (покрыто тестами `tests/test_qa_requirements.py`).
- **Режим с настоящим Keycloak (`KeycloakOidcProvider`)** включается переменными окружения, код вызова не меняется:

```
FSP_OIDC_ISSUER=https://id.fsp-russia.com/realms/fsp
FSP_OIDC_CLIENT_ID=fsp-career
FSP_OIDC_CLIENT_SECRET=...
```

Discovery через `/.well-known/openid-configuration`, обмен кода на `token_endpoint`, проверка `id_token` по `jwks_uri` (RS256, `aud`, `iss`). Поднимать Keycloak в Docker Compose по ответам постановщиков не требуется.

2. Реестр достижений

Предлагаемый контракт (`backend/app/integrations/fsp.py`):

```
GET /participants/{fsp_id}               → { full_name, region, sport_rank }
GET /participants/{fsp_id}/achievements  → [{ event_name, discipline, level, event_date, place,
                                             participants, role_in_team, skills[], source_url }]
```

`MockFspClient` выдаёт детерминированный синтетический реестр: один и тот же ID всегда даёт одни и те же данные. ID, оканчивающиеся на `0`, — участники без истории (обязательный

сценарий «истории нет»). `HttpFspClient` включается переменными `FSP_API_URL` и `FSP_API_TOKEN`.

3. Как достижения влияют на подбор

По ответам постановщиков:

Правило	Реализация
Грейд даёт только тест	Достижения не участвуют в присвоении грейда (`services/testing/service.py`)
ФСП поднимает внутри категории	Отдельный блок-бонус: закрывает долю разрыва до 100%, отсутствие ФСП не штрафуется
Все соревнования равнозначны	Уровень мероприятия (региональное, всероссийское, международное) **не взвешивается**
Профильность дисциплины (идея постановщиков)	Коэффициент 0.6–1.0 по паре «дисциплина — специализация»: ИБ-специалисту CTF важнее робототехники, бэкендеру — продуктовое и алгоритмическое программирование (`catalog.FSP_DISCIPLINE_RELEVANCE`)
Разряды	Спортивный разряд из реестра показывается работодателю рядом с бейджем ФСП, в балл не входит
Число стартов и результативность	Победа 1.0, призовое место 0.85, топ-10 0.55, участие 0.2
Нельзя набить профиль онлайн-явками	Суммарный вклад участий без высокого места ограничен 0.5
Свежий результат весит больше	−15% за каждый год давности, но не ниже 50%
Вклад в командный хакатон не выделяется	Роль в команде показывается работодателю, но в балл не входит
Задачи прошлых соревнований	Не используются в тесте, поэтому участники, решавшие их раньше, не получают преимущества

Работодатель видит строку вида «3 мероприятия (побед: 1, призовых: 1), лучшее — «…», 1 место». Кандидат может скрыть достижения настройкой приватности.

5. Описание API

Полная спецификация — OpenAPI 3: после запуска откройте Swagger `http://localhost:8000/docs` (в Docker также `http://localhost:8080/docs`) или файл openapi.json в этом каталоге. Схемы запросов и ответов описаны в `backend/app/api/schemas.py`.

Общее. Все пути начинаются с `/api`. Авторизация — заголовок `Authorization: Bearer <JWT>`. Ошибки возвращаются как `{"detail": "текст"}`: 401 — нет или недействителен токен, 403 — email не подтверждён, 404 — объект не найден или не принадлежит пользователю, 409 — конфликт состояния, 422 — ошибка данных, 429 — сработал кулдаун или лимит приглашений.

Аутентификация

Метод	Путь	Назначение
POST	`/auth/register`	Регистрация по email. Согласие на обработку ПДн (152-ФЗ) обязательно
POST	`/auth/verify-email`	Подтверждение email по токену из письма
POST	`/auth/login`	Вход по email и паролю
GET	`/auth/me`	Текущий пользователь
POST	`/auth/demo`	Только `DEV_MODE`: новый чистый аккаунт соискателя или работодателя для проверки жюри
GET	`/auth/fsp-id/config`	Куда отправить пользователя для входа через ФСП ID (mock или Keycloak)
POST	`/auth/fsp-id/mock-authorize`	Экран входа имитированного ФСП ID → одноразовый `code`
POST	`/auth/fsp-id/callback`	Обмен `code` на профиль ФСП ID: вход, регистрация или привязка + импорт достижений

Публичные

Метод	Путь	Назначение
GET	`/catalog`	ИТ-направления, 8 специализаций, 6 грейдов, навыки, опросник самооценки
GET	`/stats`	Обезличенные итоговые цифры платформы для главной страницы
GET	`/catalog/item-bank`	Состав банка семейств заданий (без ответов)
GET	`/vacancies`	Опубликованные вакансии работодателей платформы (кроме приостановленных компаний)
POST	`/integrations/huntflow/webhook/{secret}`	Вебхук Huntflow о смене этапа: `{"invitation_id": 12, "stage": "Оффер"}`
GET	`/health`	Проверка живости

Кандидат (/candidate)

Метод	Путь	Назначение
GET, PUT	`/profile`	Профиль. Дата рождения и согласие законного представителя для 15–17 лет
GET	`/profile/pdf`	Стандартизированный PDF-профиль (категория, навыки, ФСП, история тестов)
GET	`/grade-hint`	ML-подсказка грейда

		(распределение по ступеням) и вероятные специализации по тексту профиля. Только подсказка
PUT	`/privacy`	Приватность и согласие на публикацию; `show_unconfirmed` — показывать ли профиль до подтверждения грейда; `hide_below_expectation` — не принимать вилки ниже ожиданий
POST	`/resume/parse`	Распознавание полей из PDF-резюме, без сохранения; навыки по сходству написания и ML-специализация
POST	`/survey`	Опрос: ИТ-направление, специализация, заявленный грейд, ответы самооценки `answers.self`
GET, POST	`/assessments`	История попыток / старт теста на грейд (кулдауны → 429)
GET	`/assessments/{id}`	Текущее задание и сводка
POST	`/assessments/{id}/answer`	Ответ + сигналы честности (`focus_lost`, `copy_attempts`)
GET, POST	`/skill-checks`	Навыки из резюме и короткая проверка навыка (грейд не меняет)
POST, DELETE	`/fsp/link`	Прямая привязка и отвязка ФСП ID (основной путь — OIDC-callback)
GET	`/invitations`, `/invitations/{id}`	Приглашения со снимком соответствия, доверием к работодателю (`company_trust`), флагами вилки
POST	`/invitations/{id}/complaint`	Жалоба на приглашение; 3 жалобы за 30 дней приостанавливают компанию
POST	`/invitations/{id}/accept`, `/decline`	Ответ на приглашение
POST	`/invitations/{id}/revoke-contacts`, `/restore-contacts`	Отзыв и возврат доступа к контактам
GET, POST	`/invitations/{id}/messages`	Переписка по приглашению
POST	`/vacancies/{id}/apply`, GET `/applications`	Отклик на вакансию (вспомогательный путь)
GET	`/micro-tasks`	Задания работодателей для своей категории (и соседних ступеней); `recommended` — адресное задание для кандидата
GET	`/badges`	Счётчики для меню: новые приглашения и сообщения, новые задания
POST	`/micro-tasks/{id}/answer`	Ответ. Для задания с эталоном сразу возвращается `result: correct / wrong`

Работодатель (/employer)

Метод	Путь	Назначение
GET, PUT	`/company`	Профиль компании; `inn_valid` — добровольная метка доверия (контрольная сумма ИНН)
GET	`/trust`	Доверие к компании так, как его видит кандидат, и суточный лимит приглашений
GET, PUT, DELETE	`/ats`	Подключение Huntflow (без токена — демо-режим), URL вебхука, журнал синхронизации
POST	`/ats/import-vacancies`	Импорт вакансий из ATS черновиками потребностей
GET	`/badges`	Счётчики для меню: непрочитанные сообщения, новые кандидаты, новые отклики

POST	`/needs/parse`	Разбор свободного описания потребности
GET, POST, PUT	`/needs`, `/needs/{id}`	Потребности (с флагом `is_published` — открытая вакансия); `new_candidates` — новые в категории
POST	`/needs/{id}/seen`	Подборка просмотрена: следующие новые кандидаты будут отмечены
GET	`/scoring/defaults`, PUT `/needs/{id}/scoring`	Настройки модели релевантности
GET	`/needs/{id}/matches`	Объяснимая подборка: категории, блоки, требования, флаги, отсеянные фильтрами
GET	`/candidates`, `/candidates/{id}`	Поиск по банку и карточка кандидата
GET	`/candidates/{id}/pdf`	PDF-профиль кандидата: обезличенный, пока контакты не раскрыты
GET	`/needs/{id}/test-blueprint`	Как тестируются кандидаты под эту потребность
GET, POST, DELETE	`/needs/{id}/shortlist[/{cid}]`	Шорт-лист
GET, POST	`/invitations`	Адресные приглашения: суточный лимит по доверию, несколько позиций одному кандидату без дублей
POST	`/invitations/{id}/withdraw`	Отзыв приглашения
POST	`/invitations/{id}/offer`, `/hired`	Оффер после принятия и выход на работу
GET, POST	`/invitations/{id}/messages`	Переписка
GET, PATCH	`/needs/{id}/applications`, `/applications/{id}`	Отклики
GET, POST	`/micro-tasks`	Регулярные задания: `answer_type = open / exact`, `reference_answer` для автопроверки
GET	`/micro-tasks/{id}/submissions`	Ответы (`auto_checked` — проверено системой)
POST	`/micro-tasks/submissions/{id}/rate`	Оценка открытого ответа 1–5

Пример: сквозной сценарий

```
# кандидат без регистрации (DEV_MODE)
TOKEN=$(curl -s -X POST localhost:8000/api/auth/demo -H 'content-type: application/json' \
  -d '{"role":"candidate"}' | jq -r .access_token)
curl -s -X POST localhost:8000/api/candidate/survey -H "Authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' \
  -d '{"industry":"development","specialization":"backend","claimed_grade":"middle"}'
curl -s -X POST localhost:8000/api/candidate/assessments -H "Authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{}'
```

6. Библиотеки и компоненты с версиями

Версии бэкенда закреплены в `backend/requirements.txt`, фронтенда — в `frontend/package-lock.json` (сборка в Docker идёт через `npm ci`, то есть ровно по lock-файлу).

Среда

Компонент	Версия	Где задана
Python	3.13 (образ `python:3.13-slim`)	`backend/Dockerfile`
Node.js	24 (образ `node:24-alpine`)	`frontend/Dockerfile`
PostgreSQL	16 (образ `postgres:16-alpine`)	`docker-compose.yml`
nginx	1.27 (образ `nginx:1.27-alpine`)	`frontend/Dockerfile`
Mailpit	1.21 (образ `axllent/mailpit:v1.21`)	`docker-compose.yml`

Бэкенд

Пакет	Версия	Зачем
fastapi	0.128.0	HTTP API и OpenAPI
uvicorn[standard]	0.40.0	ASGI-сервер
SQLAlchemy	2.0.48	ORM
psycopg[binary]	3.2.9	драйвер PostgreSQL
pydantic	2.12.5	схемы и валидация
email-validator	2.2.0	проверка email
PyJWT	2.10.1	JWT, OIDC-код имитации, проверка id_token Keycloak по JWKS
bcrypt	4.3.0	хэши паролей
python-multipart	0.0.20	загрузка PDF-резюме
pypdf	5.4.0	извлечение текста из PDF
fpdf2	2.8.9	генерация стандартизированного PDF-профиля
numpy	2.4.1	IRT/CAT, калибровка, валидация
httpx	0.28.1	клиенты реестра ФСП, Keycloak и Huntflow
scikit-learn	1.7.2	TF-IDF, логистическая регрессия: специализация по тексту, нечёткие навыки, подсказка грейда
pytest	8.3.5	тесты

Фронтенд

Пакет	Версия (lock)	Зачем
react, react-dom	19.3.0	интерфейс
react-router-dom	7.18.4	маршрутизация, переходы через View Transitions API
typescript	6.0.3	типизация
vite	8.3.2	сборка и dev-сервер
@vitejs/plugin-react	6.1.1	JSX и fast refresh
oxlint	1.86.0	линтер
@types/react, @types/react-dom	19.3.0	типы
@types/node	24.19.1	типы конфигурации Vite

Для PDF-профиля шрифт JetBrains Mono (SIL OFL 1.1) лежит в `backend/app/assets/fonts` вместе с лицензией. В веб-интерфейсе шрифты подключаются из Google Fonts: **JetBrains Mono** — фирменный шрифт ФСП по брендбуку (SIL Open Font License 1.1), **Inter** — основной текст. Иконки — inline SVG без внешних библиотек. Анимации и тёмная тема написаны на чистом CSS (без UI-фреймворков) и учитывают `prefers-reduced-motion`.

Модели и внешние сервисы

Модель	Библиотека	Обучение	Где используется
Классификатор специализации (TF-IDF char 3–5 + LogisticRegression)	scikit-learn 1.7.2	при старте, на синтетическом корпусе резюме, < 1 с	подсказка в опросе, разбор PDF-резюме и описания вакансии
Нечёткое распознавание навыков (TF-IDF char 2–4, ближайший сосед)	scikit-learn 1.7.2	словарь навыков и транслит-синонимов	разбор резюме и вакансии: «Постгрес» → PostgreSQL
Подсказка грейда (мультиклассовая LogisticRegression)	scikit-learn 1.7.2	синтетическая популяция с известным уровнем	экран выбора грейда
Калибровка процента (изотоническая регрессия)	numpy	скрипт валидации, `calibration.json`	поле `probability` в подборке

Внешние LLM в рабочем контуре не используются: персональные данные никуда не передаются, GPU не требуется. Генерация заданий — параметрические шаблоны с программно вычисляемым ответом. Huntflow API v2 вызывается только при подключённом токене работодателя; без токена работает демонстрационный клиент без сети.

7. Сборка, развёртывание и локальный запуск

Запуск через Docker

```
docker compose up --build
```

- Приложение: http://localhost:8080 (порт меняется переменной `FRONTEND_PORT`, например в файле `.env`: `FRONTEND_PORT=8090` и `FRONTEND_URL=http://localhost:8090`)
- Swagger: http://localhost:8000/docs
- Письма подтверждения (Mailpit): http://localhost:8025

Демо-данные загружаются автоматически. Пароль у всех демо-аккаунтов: `demo12345`.

Аккаунт	Роль
`candidate@demo.ru`	кандидат без теста, чтобы пройти путь с нуля
`cand3@demo.ru`	кандидат с категорией, достижениями ФСП, приглашениями и перепиской
`employer@demo.ru`	работодатель «Цифровой Банк»: вакансии, приглашения до оффера и выхода, подключённый Huntflow (демо-режим)
`cand0@demo.ru` … `cand499@demo.ru`	500 кандидатов 8 специализаций и 6 грейдов; у части есть ФСП, у части грейд не подтверждён, `cand7` — 16 лет
`hr@retail.demo.ru`, `hr@mobilab.demo.ru`, `hr@datapulse.demo.ru`, `hr@cybershield.demo.ru` и др.	другие работодатели (всего 10)
`start.career.hr@mail.demo.ru`	компания, приостановленная по жалобам кандидатов (демонстрация антифрода)

Без регистрации: на странице входа есть «Быстрый вход» — создаёт чистый аккаунт соискателя или работодателя, чтобы самому пройти тест и подбор с нуля. **Вход через ФСП ID** — имитация Keycloak: логин вида `FSP-102347` (ID на `0` — участник без истории) или email, пароль `demo12345`. Подробнее — docs/06-fsp-integration.md.

Обновление кода не требует пересоздавать базу: недостающие колонки добавляются при старте (`ensure_columns`).

Локальный запуск без Docker

Требуются Python 3.12+ и Node 20+.

```
cd backend
pip install -r requirements.txt
python -m app.seed.demo            # SQLite-файл fsp_talent.db
uvicorn app.main:app --port 8000

cd ../frontend
npm install
npm run dev                         # http://localhost:5173, /api проксируется на :8000
# другой порт бэкенда: API_TARGET=http://127.0.0.1:8001 npm run dev
```

Без SMTP ссылка подтверждения пишется в лог бэкенда, а в `DEV_MODE=1` токен приходит в ответе регистрации (на экране есть кнопка «подтвердить сразу»).

Новые поля моделей добавляются в существующую базу автоматически при старте. Чтобы получить свежие демо-данные, удалите `backend/fsp_talent.db` (или выполните `docker compose down -v`) и снова запустите `python -m app.seed.demo`.

Тесты и валидация

```
cd backend
python -m pytest -q          # сквозной сценарий ТЗ, требования из Q&A, оффер, жалобы, ATS, ML (26 тестов)
```

```
python -m scripts.validate   # симуляция: пишет docs/02-validation.md и
app/services/matching/calibration.json
```

Нужен Linux-сервер с Docker и Docker Compose. Хватит 2 ГБ памяти.

Первый запуск

```
git clone <репозиторий> fsp-hackathon
cd fsp-hackathon
export JWT_SECRET="$(openssl rand -hex 32)"
docker compose up -d --build
```

После запуска:

Адрес	Что там
`http://<сервер>:8080`	приложение
`http://<сервер>:8000/docs`	Swagger
`http://<сервер>:8025`	Mailpit: перехваченные письма подтверждения email

Демо-данные загружаются автоматически при старте бэкенда, если их ещё нет: 500 кандидатов, 10 компаний (одна из них приостановлена по жалобам — для демонстрации антифрода), 16 вакансий, задания и приглашения в разных статусах. Пароль у демо-аккаунтов: `demo12345`.

Переменные окружения бэкенда

Переменная	По умолчанию	Назначение
`JWT_SECRET`	`change-me-in-prod`	секрет подписи токенов, **обязательно поменять**
`DEV_MODE`	`1`	`1` — токен подтверждения email возвращается в ответе регистрации (удобно для демо)
`DATABASE_URL`	Postgres из compose	строка подключения SQLAlchemy
`SMTP_HOST`, `SMTP_PORT`	`mail`, `1025`	почтовый сервер (в compose — Mailpit)
`FRONTEND_URL`	`http://localhost:8080`	адрес фронтенда для ссылки в письме; поменять на публичный
`CORS_ORIGINS`	localhost	разрешённые источники через запятую; добавить публичный адрес
`FSP_API_URL`, `FSP_API_TOKEN`	не заданы	реальный API реестра ФСП; без них работает mock
`GRADE_CHANGE_COOLDOWN_DAYS`	`90`	как часто можно менять подтверждённый грейд
`INVITATION_TTL_DAYS`	`14`	срок жизни приглашения
`FRONTEND_PORT`	`8080`	внешний порт фронтенда в compose (если 8080 занят — например, `FRONTEND_PORT=8090`)

Для публичного адреса задайте `FRONTEND_URL` и `CORS_ORIGINS` в `docker-compose.yml` или в файле `.env` рядом с ним.

Обновление

```
git pull
docker compose up -d --build
```

Миграций пока нет: схема создаётся при старте. Если модели данных менялись, пересоздайте базу. Это удалит все данные, демо-данные загрузятся заново.

```
docker compose down -v
docker compose up -d --build
```

Проверка

```
curl http://localhost:8000/api/health        # {"status":"ok"}
docker compose logs -f backend
```

8. Соответствие ТЗ и ответам постановщиков

Каждый пункт сессии вопросов и ответов (Q&A «Лидеры цифровой трансформации 2026», задача ФСП) сопоставлен с реализацией. Автотесты на эти требования — `backend/tests/test_qa_requirements.py` и `backend/tests/test_e2e.py`.

Обязательный минимум MVP

Требование	Как выполнено	Где
Кабинеты кандидата и работодателя	Два кабинета с разграничением ролей (JWT, `require_candidate` / `require_employer`)	`frontend/src/pages/*`, `backend/app/api/candidate.py`, `employer.py`
Поиск и подбор	Подборка по потребности и поиск по всему банку кандидатов	`GET /employer/needs/{id}/matches`, `GET /employer/candidates`
Первичный тест присваивает грейд	Адаптивный тест (IRT 2PL + CAT) по категории, 12–22 задания до 20 минут, решение по медиане апостериорного θ	`services/testing/cat.py`, `service.py`
Автоматическая категоризация	Категория «специализация · грейд» по итогам теста; работодатель сначала видит карточки категорий, затем кандидатов	`ranking.group_categories`, `NeedPage.tsx`
Объяснимый подбор	Процент, откалиброванная вероятность, 6 блоков, покрытие каждого требования, флаги	`ranking.score_candidate`, `match.tsx`
Адресное приглашение	Позиция, вилка, описание, способ связи, снимок соответствия, суточный лимит по доверию, срок 14 дней, статусы до оффера и выхода	`POST /employer/invitations`
Одинаковые задачи не расходятся между кандидатами	Параметрические семейства: числа, имена, код и порядок ответов генерируются заново, ответ вычисляется программно и не уходит на клиент	`services/testing/item_bank.py`
Документация с обязательным перечнем	См. раздел «Документация» ниже	`docs/`
Десятки одновременных пользователей	Нагрузочная проверка: 60 запросов в 50 потоков — 0 ошибок, p95 1,4 с	05-architecture.md
Профиль: форма или PDF с автораспознаванием; автогенерация стандартизированного PDF-профиля	Разбор PDF-резюме в поля; генерация PDF по единому шаблону (`GET /candidate/profile/pdf`), работодателю — обезличенная версия до согласия	`services/profile_pdf.py`, кнопки «Скачать PDF-профиль» и «PDF»
Правила видимости данных	Обезличенные карточки, контакты только после согласия, настройки приватности	`api/common.py`

Тест, грейд, вакансия

Ответ постановщиков	Реализация	
Грейд только по первичному тесту категории, не по вакансии и не по ФСП	Грейд меняет только `service.finish` по итогам теста на грейд. Проверка навыка и ФСП грейд не меняют	
Достижения ФСП — ранжирование внутри категории	Блок-бонус ФСП после агрегации, сортировка сначала по точности категории	
Неподтверждённый грейд показывать со статусом и опускать, не скрывать	Профиль остаётся в выдаче с заявленным грейдом и пометкой «не подтверждён», штраф ×0.6 и	

	сортировка ниже подтверждённых в своей категории; пунктирная рамка в карточке	
Кандидат сам выбирает грейд, подсказка по резюме допустима	Выбор в опросе из 6 ступеней. `GET /candidate/grade-hint` — ML-подсказка по стажу, стеку, призовым местам ФСП и опроснику самооценки, с пометкой «это не оценка»	
Неподтверждённый грейд: показывать или скрывать с обоснованием	По умолчанию показывается; кандидат сам может скрыть профиль до пересдачи (`privacy.show_unconfirmed`)	
Повышение — отдельным тестом	При уверенном подтверждении (`upgrade_eligible`) система сама предлагает тест на ступень выше, доступный сразу (30 дней). Автоповышения нет	
Кулдаун; провал и уход ниже без сжигания попытки	Смена подтверждённого грейда — раз в 90 дней; повтор того же уровня после неудачи — через 30 дней; тест на уровень ниже — сразу. Неподтверждённый грейд кулдаун не запускает	
«Отрасль» = IT-направление	6 ИТ-направлений и 8 специализаций: Backend, Frontend, Mobile, Data/ML, Аналитик, QA, DevOps, ИБ. Грейды Intern, Junior, Junior+, Middle, Middle+, Senior	
Грейд — уровень навыков, а не стаж (ответ VS.1)	Грейд присваивает тест; стаж — атрибут профиля и признак для подсказки	
Тест единый под категорию, но не одинаковый пул (VS.5)	Пул семейств по специализации, варианты генерируются заново, адаптивный выбор из топ-4	
Регулярные задания адресно (E09)	Система выбирает для кандидата «задание для вас» по навыку с наименьшей уверенностью; счётчик новых заданий в меню	
Задания работодателя: эталон проверяет система, открытый ответ — работодатель	`answer_type = exact` с `reference_answer` (варианты через «	», нормализация регистра и пробелов) проверяется сразу; `open` оценивает работодатель 1–5. Оценки становятся доказательством навыков. Экран «Задания»
Импорт чужих вакансий не нужен	Не реализуется: вакансии создают только работодатели платформы	
Зарплата: гросс или нетто — подписать	Везде подписано «gross, до вычета НДФЛ»: формы, приглашения, вакансии, ожидания кандидата	

LLM, песочница, антисписывание

Ответ постановщиков	**Реализация**
LLM можно (в т.ч. зарубежные), использование описать; ML и NLP — преимущество	**Внешних LLM в рабочем контуре нет.** ML/NLP на scikit-learn без GPU: специализация по тексту, нечёткие навыки, подсказка грейда, изотоническая калибровка (08-libraries.md). Офлайн-роль LLM (новые семейства с калибровкой как pretest) описана в 01-concept.md
GPU на стенде нет	Всё считается на CPU
Песочница, прокторинг, защита от LLM — плюс	«Знаниевые» задания с малым числом формулировок весят в оценке вдвое меньше. Прокторинг «лайт»: уходы с вкладки, попытки копировать условие (копирование заблокировано), подозрительно быстрые ответы. Работодатель видит сводку «тест пройден чисто / есть сигналы» — как сигнал, без влияния на θ. Лимит времени на задание и запрет

	поздних ответов
Задачи прошлых соревнований ФСП — с защитой от преимущества решавших	В тест не включаются: банк состоит из собственных генерируемых семейств

ФСП и ФСП ID

Ответ постановщиков	Реализация
Подтверждённый профиль — данные из ФСП ID, а не самозаявление	Достижения импортируются только из реестра при входе или привязке через ФСП ID
Имитировать привязку и вход, скелет интеграции	OIDC Authorization Code: экран имитированного ФСП ID, одноразовый code, проверка state; `KeycloakOidcProvider` включается переменными окружения. Схемы — 06-fsp-integration.md
Keycloak в Docker Compose не ждут	Не поднимается
Альтернативная регистрация по email	Есть, параллельно с ФСП ID
Ранжировать дисциплины не надо, все соревнования равно важны; можно учитывать профильность (E08)	Уровень мероприятия не взвешивается; коэффициент профильности дисциплины для специализации 0.6–1.0 (ИБ-специалисту CTF важнее робототехники)
Данные ФСП: участия, места, разряды, дисциплины	Импортируются мероприятие, дисциплина, дата, место, число участников, роль, навыки, спортивный разряд
Учитывать число соревнований и результативность, не давать набить профиль онлайн-явками	Победа > призовое > топ-10 > участие; вклад участий ограничен 0.5
Срок давности — по желанию	−15% за год, не ниже 50%
Вклад в командный хакатон честно не выделить	Роль в команде только отображается, в балл не входит

Подбор, контакты, аудитория

Ответ постановщиков	Реализация
Категоризация, объяснимый подбор, адресное приглашение + оригинальность	Обратная механика: работодатель выбирает категорию, потом человека; настраиваемая модель релевантности; зарплатный сигнал без раскрытия ожиданий; фильтр «не ниже моих ожиданий»; уведомления о новых кандидатах; несколько позиций одному кандидату; оффер и выход на работу
Защита от фиктивных вакансий и ATS — концепция, плюс (FTT.5)	Реализованы: оценка доверия 0–100, суточные лимиты, жалобы с автоприостановкой, флаг вилки-приманки; Huntflow API v2 — передача кандидатов и этапов, вебхук, импорт вакансий
ФИО и места работы показывать допустимо (VS.4)	До согласия карточка обезличена (выбор в пользу приватности по 152-ФЗ); контакты закрыты всегда до согласия
Несколько категорий у кандидата — плюс (VS.2)	В MVP одна категория; несколько — в плане развития
Контакты открываются пригласившему после принятия; отзыв доступа — плюс	Реализовано. Кандидат может отозвать и вернуть доступ в карточке приглашения; работодатель снова видит обезличенную карточку
Чат на платформе — плюс	Переписка по приглашению для обеих сторон, без раскрытия контактов; счётчики непрочитанного в навигации
Проверка компании — добровольная метка доверия	Проверка ИНН по контрольной сумме → метка «ИНН проверен» у кандидата. Основной сценарий не блокируется
Правовой минимум, 152-ФЗ	Согласие на обработку при регистрации и на экране ФСП ID, отдельное согласие на публикацию, отзыв в любой момент
Несовершеннолетние: с 15 лет лёгкий труд, согласие родителей	Дата рождения в профиле; до 15 лет публикация запрещена; 15–17 — только с согласием законного представителя (ФИО, контакт); работодатель видит отметку «до 18, только лёгкий труд (ст. 63 ТК РФ)»
Аудитория — и начинающие участники ФСП, и опытные специалисты	Грейды от Intern до Senior с промежуточными Junior+ и Middle+; ФСП — бонус, а не фильтр
Дизайнеры — плюс, приоритет техническим ролям (WA.2)	Приоритет техническим специализациям; для дизайнеров нужна иная форма проверки (портфолио + разбор кейса), вынесено в развитие

Проверка жюри и приёмка

Ответ постановщиков	Реализация
Жюри само проходит тесты, регистрация с нуля не обязательна, демо-режим допустим	«Быстрый вход» на странице входа создаёт чистый аккаунт соискателя или работодателя без почты; демо-аккаунты с готовыми данными
Свой набор пар и процедура валидации	Синтетическая популяция с известной истиной, прогон через реальный код, отчёт — 02-validation.md (`python -m scripts.validate`)
Разделяющая сила и воспроизводимость	Точность грейда, матрица ошибок, каппа повторного прохождения, атака «слитый ключ» — в отчёте валидации
Стенд 32–64 ГБ RAM, ~8 CPU, без GPU	Укладываемся: 4 контейнера, расчёты на CPU
Питч 10 минут, видео вместо стенда допустимо (E10)	Презентация — `docs/ФСП_Карьера_презентация.pptx` (17 слайдов по шаблону ЛЦТ-2026, команда из двух человек)
После дедлайна идею не менять	Фиксируем коммитом; упавший контейнер поднимается `docker compose up -d`

Документация (обязательный перечень)

Пункт	Файл
Архитектура	05-architecture.md
Механика тестирования	01-concept.md, раздел 2
Механика подбора и логика категоризации	01-concept.md, раздел 1
Процедура валидации и результаты	01-concept.md, раздел 3; 02-validation.md
Схемы интеграции с ФСП / ФСП ID	06-fsp-integration.md
Описание API	07-api.md, openapi.json, Swagger `/docs`
Библиотеки с версиями	08-libraries.md
Сборка и запуск	../README.md, 04-deploy.md